

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №9

«Обработка исключительных ситуаций.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Реализовать класс, перегрузить для него операции, указанные в варианте.

2. Определить исключительные ситуации.

3. Предусмотреть генерацию исключительных ситуаций.

Класс- контейнер СПИСОК с ключевыми значениями типа `int`.

Реализовать операции:

`[]` – доступа по индексу;

`int()` – определение размера списка;

`*` вектор – умножение элементов списков `a[i]*b[i]`;

`+n` - переход вправо к элементу с номером `n`.

АНАЛИЗ.

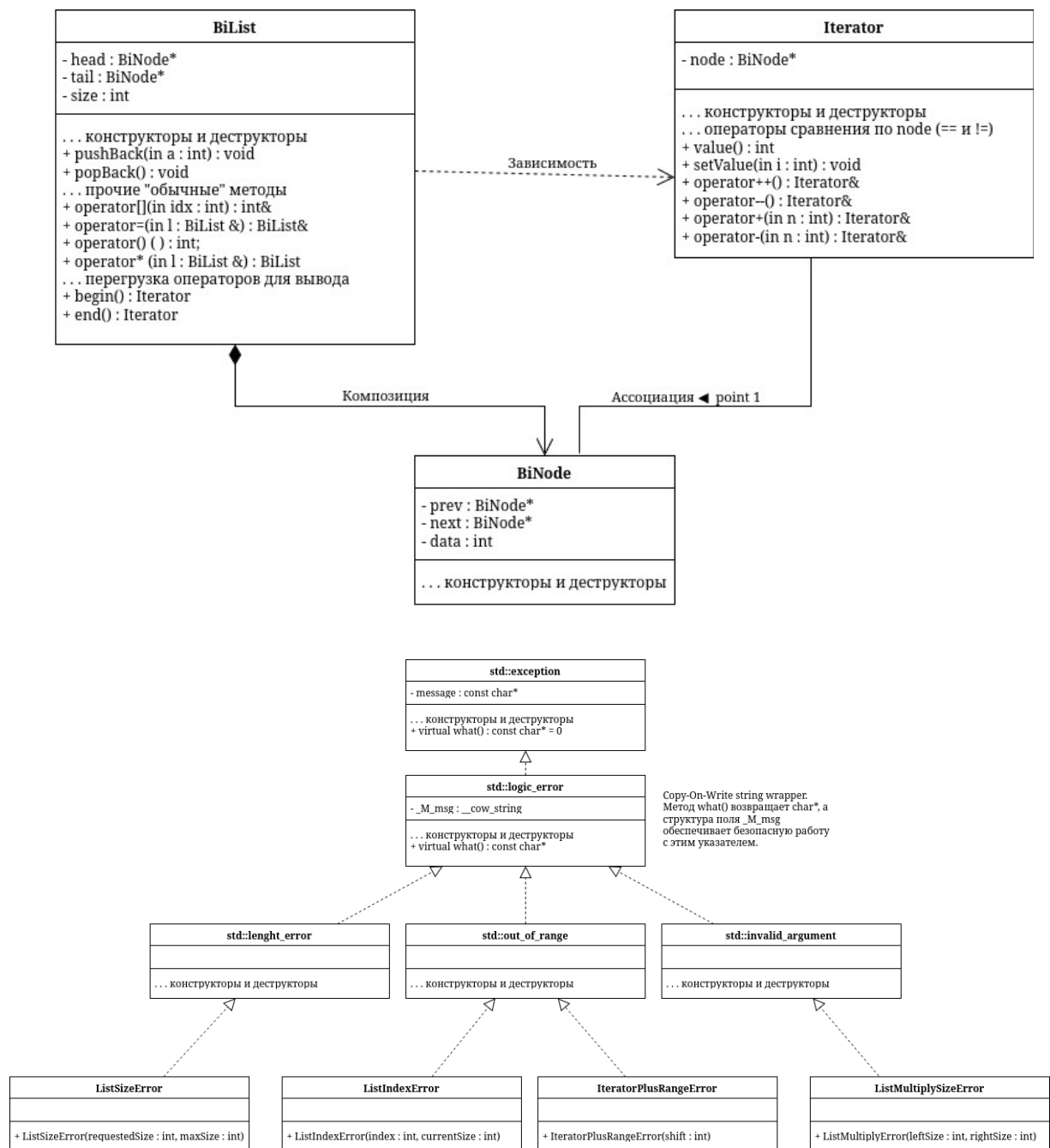
1. Создадим класс `BiNode` – узел в списке.

2. Создадим класс `BiList` – двунаправленный список.

3. Создадим класс `Iterator` для перемещения по списку.

4. Создадим классы `ListSizeError`, `ListIndexError`, `IteratorPlusRangeError`, `ListMultiplySizeError`, наследуя их от стандартных классов ошибок.

UML-ДИАГРАММА



КОД Errors.h

```

#ifndef PSTU_TEACHING_ERRORS_H
#define PSTU_TEACHING_ERRORS_H
#include <stdexcept>
#include <string>

```

```

class ListSizeError : public std::length_error {
public:
    ListSizeError(int requestedSize, int maxSize)
        : std::length_error("BiList(int): requested size " + std::to_string(requestedSize) +

```

```

        " exceeds max_size " + std::to_string(maxSize)) {}
};

class ListIndexError : public std::out_of_range {
public:
    ListIndexError(int index, int currentSize)
        : std::out_of_range("BiList::operator[]: index " + std::to_string(index) +
            " is out of range [0 ; " + std::to_string(currentSize) + ')') {}
};

class IteratorPlusRangeError : public std::out_of_range {
public:
    IteratorPlusRangeError(int shift)
        : std::out_of_range("Iterator::operator+: invalid shift " + std::to_string(shift) +
            " (shift moves iterator past end)") {}
};

class ListMultiplySizeError : public std::invalid_argument {
public:
    ListMultiplySizeError(int leftSize, int rightSize)
        : std::invalid_argument("BiList::operator*: size mismatch (left=" + std::to_string(leftSize)
+
            ", right=" + std::to_string(rightSize) + ")") {}
};

#endif // PSTU_TEACHING_ERRORS_H

```

KOD Bidirectional.h

```

#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H
#include <iosfwd>

class BiList;
class Iterator;

class BiNode {
    friend BiList;
    friend Iterator;
    BiNode *prev;
    BiNode *next;
    int data;

    BiNode() { prev = nullptr, next = nullptr, data = 0; }
}

```

```
};
```

```
class Iterator {  
    BiNode *node;  
  
public:  
    Iterator() { node = nullptr; }  
    Iterator(BiNode *p) { node = p; }  
    Iterator(const Iterator &it) { node = it.node; }  
    bool operator==(const Iterator &it) { return node == it.node; }  
    bool operator!=(const Iterator &it) { return node != it.node; }  
  
    Iterator &operator++();  
    Iterator &operator--();  
    Iterator &operator+(int);  
    Iterator &operator-(int);  
  
    int& operator*() { return node->data; }  
    int value() { return node->data; }  
    void setValue(int i) { node->data = i; }  
};
```

```
class BiList {  
    const int max_size = 100;  
    BiNode *head;  
    BiNode *tail;  
    int size;  
  
public:  
    BiList();  
    BiList(int);  
    BiList(BiList &);  
    ~BiList() { clear(); }  
  
    void pushBack(int);  
    void popBack();  
    void pushFront(int);  
    void popFront();  
    void insert(int, int);  
    void remove(int);  
    int find(int &);  
    void clear();  
  
    int &operator[](int);  
    BiList &operator=(BiList &);
```

```

int operator() { return size; }
BiList operator*(BiList &);

friend std::ostream &operator<<(std::ostream &, BiList);
friend std::istream &operator>>(std::istream &, BiList &);

Iterator begin() { return Iterator(head); }
Iterator end() { return Iterator(tail->next); }
};

#endif ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_BI_H

```

KOД Bidirectional.cpp

```

#include "bidirectional.h"
#include "errors.h"
#include <iostream>
#include <fstream>

void BiList::pushBack(int data) {
    auto node = new BiNode;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }

    node->prev = tail;
    tail->next = node;
    tail = node;
    size++;
}

void BiList::popBack() {
    if (size == 0)
        return;

    auto del = tail;
    tail = tail->prev;

    if (tail != nullptr)

```

```
    tail->next = nullptr;
else
    head = nullptr;

delete del;
size--;

}
```

```
void BiList::pushFront(int data) {
    auto node = new BiNode;
    node->data = data;
    if (size == 0) {
        head = node;
        tail = node;
        size++;
        return;
    }
```

```
    node->next = head;
    head->prev = node;
    head = node;
    size++;

}
```

```
void BiList::popFront() {
    if (size == 0)
        return;

    auto del = head;
    head = head->next;

    if (head != nullptr)
        head->prev = nullptr;
    else
        tail = nullptr;

    delete del;
    size--;
}
```

```
}
```

```
void BiList::insert(int idx, int data) {  
    if (idx == size) {  
        pushBack(data);  
        return;  
    }  
    if (idx == 0) {  
        pushFront(data);  
        return;;  
    }  
  
    if (!(0 < idx && idx < size)) {  
        return;  
    }  
  
    auto node = head;  
    for (int i = 0; i < idx; i++) {  
        node = node->next;  
    }  
  
    auto newNode = new BiNode;  
    newNode->data = data;  
  
    node->prev->next = newNode;  
    newNode->prev = node->prev;  
  
    node->prev = newNode;  
    newNode->next = node;  
  
    size++;  
}
```

```
void BiList::remove(int idx) {  
    if (!(0 <= idx && idx < size)) {  
        return;  
    }  
    if (idx == 0) {  
        popFront();  
        return;  
    }
```



```

    }
    if (idx == size - 1) {
        popBack();
        return;
    }

    auto node = head;
    for (int i = 0; i < idx; i++) {
        node = node->next;
    }

    node->next->prev = node->prev;
    node->prev->next = node->next;

    delete node;
    size--;
}

```

```

int BiList::find(int &data) {
    auto node = head;
    int i = 0;
    while (node != nullptr) {
        if (node->data == data) {
            return i;
        }
        i++;
        node = node->next;
    }

    return -1;
}

```

```

void BiList::clear() {
    auto node = head;
    while (node != nullptr) {
        auto tmp = node->next;
        delete node;
        node = tmp;
    }
    head = nullptr;
    tail = nullptr;
    size = 0;
}

```

```
}
```

```
int &BiList::operator[](int idx) {  
    if (!(0 <= idx && idx < size)) {  
        throw ListIndexError(idx, size);  
    }  
}
```

```
    auto node = head;  
    for (int i = 0; i < idx; i++) {  
        node = node->next;  
    }  
}
```

```
    return node->data;  
}
```

```
BiList &BiList::operator=(BiList &l) {  
    if (this == &l)  
        return *this;  
  
    this->clear();  
    for (int i = 0; i < l.size; i++) {  
        this->pushBack(l[i]);  
    }  
}
```

```
    return *this;  
}
```

```
BiList BiList::operator*(BiList &l) {  
    if (this->size != l.size) {  
        throw ListMultiplySizeError(this->size, l.size);  
    }  
}
```

```
    BiList res;  
    for (int i = 0; i < this->size; i++) {  
        res.pushBack((*this)[i] * l[i]);  
    }  
    return res;  
}
```

```
BiList::BiList() {  
    head = nullptr;  
}
```

```

    tail = nullptr;
    size = 0;
}

BiList::BiList(int s) : BiList() {
    if (s > max_size) {
        throw ListSizeError(s, max_size);
    }

    for (int i = 0; i < s; i++)
        pushBack(0);

}

BiList::BiList(BiList &l) : BiList() {
    for (int i = 0; i < l.size; i++) {
        pushBack(l[i]);
    }
}

std::ostream &operator<<(std::ostream &stream, BiList l) {
    for (int i = 0; i < l(); i++)
        stream << l[i] << '\t';
    stream << '\n';
    return stream;
}

std::istream &operator>>(std::istream &stream, BiList &l) {
    std::cout << "Size? ";
    int s;
    stream >> s;
    std::cout << "Elements? ";
    for (int i = 0; i < s; i++) {
        int a;
        stream >> a;
        l.pushBack(a);
    }
    return stream;
}

Iterator & Iterator::operator+(int a) {
    for (int i = 0; i < a; i++) {
        if (node == nullptr) {
            throw IteratorPlusRangeError(a);

```

```

    }
    node = node->next;
}
return *this;
}

Iterator & Iterator::operator-(int a) {
    for (int i = 0; i < a; i++)
        node = node->prev;
    return *this;
}

Iterator & Iterator::operator--() {
    node = node->prev;
    return *this;
}

Iterator & Iterator::operator++() {
    node = node->next;
    return *this;
}

```

КОД main.cpp

```

#include <iostream>
#include "bidirectional.h"
#include "errors.h"

using namespace std;

int main() {
    try {
        BiList list(101);
    } catch (const exception &e) {
        cout << e.what() << '\n';
    }

    BiList a(3), b(2);

    try {
        cout << a[3] << '\n';
    } catch (const exception &e) {
        cout << e.what() << '\n';
    }
}

```

```

try {
    BiList c = a * b;
    cout << c;
} catch (const exception &e) {
    cout << e.what() << '\n';
}

try {
    Iterator it = a.begin();
    it + 10;
    cout << it.value() << '\n';
} catch (const exception &e) {
    cout << e.what() << '\n';
}

return 0;
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРОГРАММЫ

BiList(int): requested size 101 exceeds max_size 100

BiList::operator[]: index 3 is out of range [0 ; 3)

BiList::operator*: size mismatch (left=3, right=2)

Iterator::operator+: invalid shift 10 (shift moves iterator past end)

ВОПРОСЫ

1. Что представляет собой исключение в C++?

Исключение – это непредвиденное или аварийное событие.

В C++ исключение – это объект, который система должна генерировать при возникновении исключительной ситуации. Генерация такого объекта и создает исключительную ситуацию.

2. На какие части исключения позволяют разделить вычислительный процесс? Достоинства такого подхода?

Исключения позволяют разделить вычислительный процесс на 2 части:

- 1) обнаружение аварийной ситуации (неизвестно как обрабатывать);
- 2) обработка аварийной ситуации (неизвестно, где она возникла).

Достоинства такого подхода:

- 1) удобно использовать в программе, которая состоит из нескольких модулей
- 2) не требуется возвращать значение в вызывающую функцию.

3. Какой оператор используется для генерации исключительной ситуации?

Исключение генерируется оператором `throw <выражение>`, где `<выражение>`

- либо константа,
- либо переменная некоторого типа,
- либо выражение некоторого типа.

4. Что представляет собой контролируемый блок? Для чего он нужен?

Для проверки возникновения исключения используется контролируемый блок `try{}`, с которым связана одна или несколько секций-ловушек `catch`. Все переменные, объявленные в этом блоке, являются локальными для этого блока.

Нужен для того, чтобы охватить участок кода, в котором может возникнуть исключительная ситуация, и связать его с обработчиками.

5. Что представляет собой секция-ловушка? Для чего она нужна?

Секция-ловушка — это блок `catch`, связанный с контролируемым блоком `try{}`. Форма записи секции-ловушки следующая: `catch( спецификация исключения )`.

Нужна для перехвата и обработки исключения. При отсутствии подходящей секции-ловушки осуществляется вызов стандартной функции завершения `terminate()`.

6. Какие формы может иметь спецификация исключения в секции-ловушке? В каких ситуациях используются эти формы?

Спецификация исключения может иметь три формы:

- 1) (тип имя)
- 2) (тип)
- 3) (...)

Формы 1 и 2 обрабатывают конкретные исключения, а форма 3 перехватывает все исключения, такую ловушку надо помещать последней, тогда она будет обрабатывать

все исключения, которые ещё не были обработаны.

Форма 1 означает, что объект передаётся в блок обработки, чтобы его каким-то образом там использовать, например, для вывода информации в сообщении об ошибке.

Примеры:

```
catch( exception e)
```

```
catch( exception &e)
```

```
catch( const exception &e)
```

```
catch( exception *e)
```

7. Какой стандартный класс можно использовать для создания собственной иерархии исключений?

Можно определять собственные исключения, унаследовав их от класса exception.

Наследование от стандартных классов позволит использовать метод what для вывода сообщений об ошибках.

8. Каким образом можно создать собственную иерархию исключений?

Для создания собственной иерархии исключений надо объявить свой базовый класс-исключение/

9. Если спецификация исключений имеет вид: void f1()throw(int,double); то какие исключения может порождать функция f1()?

В нынешнем стандарте C++ такой синтаксис и логика не поддерживается, но в старых стандартах функция f1() может порождать исключения типов int и double.

10. Если спецификация исключений имеет вид: void f1()throw(); то какие исключения может порождать функция f1()?

В нынешнем стандарте C++ такой синтаксис и логика не поддерживается, но в старых стандартах считается, что функция исключений не генерирует. Но работает поехерт.

11. В какой части программы может генерироваться исключение?

Исключение генерируется внутри контролируемого блока try{}

12. Функция вычисления площади треугольника по формуле Герона в 4 вариантах.

```
class TriangleException : public std::exception {
public:
    const char* what() const noexcept override {
        return "Invalid triangle sides";
    }
};
```

```
double heronNoSpec(double a, double b, double c) {
    if (a <= 0 || b <= 0 || c <= 0 ||
        a + b <= c || a + c <= b || b + c <= a)
        throw -1;
    double s = (a + b + c) / 2.0;
```

```

    return sqrt(s * (s-a) * (s-b) * (s-c));
}

double heronNoThrow(double a, double b, double c) noexcept {
    if (a <= 0 || b <= 0 || c <= 0 ||
        a + b <= c || a + c <= b || b + c <= a)
        return -1.0;
    double s = (a + b + c) / 2.0;
    return sqrt(s * (s-a) * (s-b) * (s-c));
}

double heronStdSpec(double a, double b, double c)
    throw(std::runtime_error) {
    if (a <= 0 || b <= 0 || c <= 0 ||
        a + b <= c || a + c <= b || b + c <= a)
        throw std::runtime_error("Invalid triangle sides");
    double s = (a + b + c) / 2.0;
    return sqrt(s * (s-a) * (s-b) * (s-c));
}

double heronOwnSpec(double a, double b, double c)
    throw(TriangleException) {
    if (a <= 0 || b <= 0 || c <= 0 ||
        a + b <= c || a + c <= b || b + c <= a)
        throw TriangleException();
    double s = (a + b + c) / 2.0;
    return sqrt(s * (s-a) * (s-b) * (s-c));
}

```